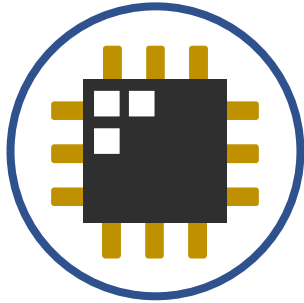
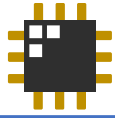




EEE 143: Switching Theory and Digital Logic Design

Lecture 3 – Boolean Algebra:
Manipulation and Minimization, Canonical forms

1



The cost of implementing logic circuits

- Implementing logic circuits comes at a cost. **Cost** could mean *power consumption*, *area* occupied by the circuit, *delay*, etc.
 - For example, cost may be assigned to a logic circuit based on how many logic gates are being used:
 - $E = [(AC \cdot W'X') + (AC' \cdot WX)] + [(AC \cdot Y'Z) + (AC' \cdot YZ')]$
 - **Cost:** 3 two-input OR gates, 12 two-input AND gates, 5 inverters

**Can we simplify Boolean expressions
to minimize cost?**

2

We can have two different Boolean expressions that actually give the same truth table. When that happens, it would be preferable to go with the simpler one. Why? Because in real life, these logic gates have corresponding hardware components that consumes power. So, the more complicated your circuit is, the more power it consumes. Therefore, if you can simplify your circuit, it would be better. But power isn't the only thing to consider. These logic gates also occupies an area as well. More gates mean more wires, and that makes your circuit messier. Lastly, we have another concern, which is the delay. Logic gates aren't ideal, signals take time to pass through them. The more gates your signal has to go through, the slower your system becomes. So, keeping things simple is about making your circuit more efficient overall.

Now, how do we measure cost? We can estimate how much the cost is by counting how many logic gates are needed to convert an expression into circuit level.

For the given example, let's go step by step:

AND Gates

Each of these expressions inside the parentheses is an AND operation:

- $AC \cdot W'X'$ – 1 two-input AND gate
- $AC' \cdot WX$ – 1 two input AND gate
- $AC \cdot Y'Z$ – 1 two-input AND gate
- $AC' \cdot YZ'$ – 1 two input AND gate

Now, each of the terms of the expression uses two-input AND gate: AC , $W'X'$, AC' , WX , AC , $Y'Z$, AC' , and YZ' .

Counting everything, we'll have 12 two-input AND gates.

OR Gates

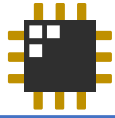
Each large term is an OR operation:

- $(AC \cdot W'X') + (AC' \cdot WX)$ - 1 two-input OR gate
- $(AC \cdot Y'Z) + (AC' \cdot YZ')$ – 1 two-input OR gate
- Then we combine the two large terms with one more OR gate – 1 two-input OR gate

Counting all OR gates used, we'll have 3 two-input OR gates.

Lastly, we also have NOT operations for W' , X' , Y' , Z' , and C' , resulting to 5 total inverters used.

So, that's how we describe the relative cost to implement this expression.

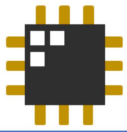


Boolean algebra

- Boolean algebra allows us to create algebraic expressions that represent logic behavior
 - Through algebraic manipulation, Boolean expressions can be written in another equivalent expression
 - Multiple Boolean expressions can describe the same truth table!
- Looks like conventional algebra (mathematics) but behaves more like ***algebra of sets/set theory***

3

Again, if we can write another Boolean expression that corresponds to the same truth table, why not? But to do that, we need to know how to simplify the expression. This is where Boolean algebra comes in. Like conventional algebra, Boolean algebra follows a structured set of rules, but it's more closely related to set theory. Our goal now is to derive different Boolean expressions that represent the same truth table.



Axiom, Postulates, and Theorems

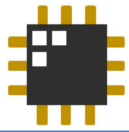
- **Axioms** and **postulates** are given facts we don't need to prove
- **Theorems** are proven using axioms/postulates
- Knowing the postulates and theorems of Boolean algebra will help us understand ***how to find alternative/equivalent expressions of a given expression***

4

Now, let's talk about the foundation of Boolean algebra. In any mathematical system, we have certain facts that we accept as true without needing proof, these are called axioms or postulates.

Now using these axioms, we can prove theorems. Theorems aren't just assumed to be true, they are proven logically through the axioms.

So, why is this important? In Boolean algebra, knowing postulates and theorems allows us to manipulate and simplify Boolean expressions. This means we can rewrite a given logic expression in a different but equivalent form.



Axioms of Boolean Algebra

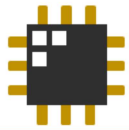
- Axioms and theorems of Boolean algebra obey the principle of **duality**.
 - **Duality Principle**
 - If the symbols 0 and 1 and the operators $\cdot$ (AND) and $+$ (OR) are interchanged, the statement will still be correct.
 - The **dual** of a statement is essentially its *negation*
 - We use the prime symbol ($'$) to denote the **dual** of a statement.

5

Now, let's talk about the Duality Principle in Boolean algebra.

Duality is essentially the negation of a statement that must always hold true. Considering this, duals always come in pairs. If there's a rule that defines how 0 behaves, there must be a corresponding rule that defines what's equivalent to 0 in the dual form.

So why does this matter? Because when we simplify expressions, knowing that every Boolean law has a dual allows us to derive new truths without having to prove them from scratch.



Axioms of Boolean Algebra

- **Axiom #1:** $B = 0$, if $B \neq 1$

- Dual: $B = 1$, if $B \neq 0$

- **Axiom #2:** $0' = 1$

- Dual: $1' = 0$

- **Axiom #3:** $0 \bullet 0 = 0$

- Dual: $1 + 1 = 1$

- **Axiom #4:** $1 \bullet 1 = 1$

- Dual: $0 + 0 = 0$

- **Axiom #5:** $0 \bullet 1 = 1 \bullet 0 = 0$

- Dual: $1 + 0 = 0 + 1 = 1$

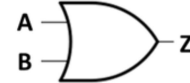
At any given time, a Boolean expression would evaluate to a 0 or a 1

- **Only two possible states!**

Defines the AND logic and OR logic



A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1



A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1

Again, axioms are facts that we don't need to prove anymore.

We grouped it into two general groups. The first two axioms are basically indicating this: at any given time, there is only two states possible for your value, it's either 0 or 1.

Axiom #1: If $B = 0$, that means B is not 1. The dual says that if $B = 1$, then B is not 0. This makes sense because Boolean logic only deals with two values: 0 and 1.

Axiom #2: The complement of 0 is 1. The dual tells us that the complement of 1 is 0, which follows directly how Boolean negation works.

The next group is basically defining what is the logic of the AND and OR.

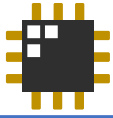
Axiom #3: $0 \bullet 0 = 0$. The dual flips AND ($\bullet$) to OR ($+$) and 0 to 1, giving us $1 + 1 = 1$.

Axiom #4: $1 \bullet 1 = 1$. Its dual is $0 + 0 = 0$, meaning OR-ing 0 with itself still results to 0 and AND-ing 1 to itself still results to 1.

Axiom #5: $0 \bullet 1 = 1 \bullet 0 = 0$. The dual swaps AND with OR, giving $1 + 0 = 0 + 1 = 1$.

Duality isn't just about flipping values, it's about preserving Boolean logic's

fundamental structure in a negated way.



Boolean theorems: Single variable

- For single variables: Let's say **A** is a variable

- $A(1) = A$ (Any variable ANDed to 1 is itself)
- $A + 0 = A$ (Any variable ORed to 0 is itself)

Identity theorem

AND logic

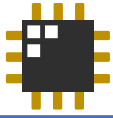
A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1

OR logic

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1

7

$A(1) = A$ It states that any variable ANDed with 1 results in the variable itself. If you look at the truth table for the AND logic, let's set B to be always 1, so now, as long as the other operand is fixed to 1, the output just copies whatever A is.



Boolean theorems: Single variable

- For single variables: Let's say **A** is a variable

- $A(1) = A$ (Any variable ANDed to 1 is itself)
- $A + 0 = A$ (Any variable ORed to 0 is itself)

Identity theorem

AND logic

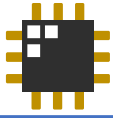
A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1

OR logic

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1

8

$A + 0 = A$: It states that any variable ORed with 0 results in the variable itself. If you look at the truth table for the OR logic, let's set B to be always 0, so now, as long as the other operand is fixed to 0, the output just copies whatever A is.



Boolean theorems: Single variable

- For single variables: Let's say **A** is a variable

- $A(0) = 0$ (Any variable ANDed to 0 is 0)
- $A + 1 = 1$ (Any variable ORed to 1 is 1)

Null theorem

AND logic

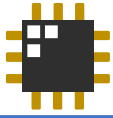
A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1

OR logic

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1

9

$A(0) = 0$: It states that any variable ANDed with 0 results in 0. If you look at the truth table for the AND logic, let's set B to be always 0, so now, as long as the other operand is fixed to 0, the output will always be 0 regardless of what A is.



Boolean theorems: Single variable

- For single variables: Let's say **A** is a variable

- $A(0) = 0$ (Any variable ANDed to 0 is 0)
- $A + 1 = 1$ (Any variable ORed to 1 is 1)

Null theorem

AND logic

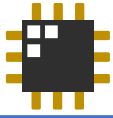
A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1

OR logic

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1

10

$A + 1 = 1$: It states that any variable ORed with 1 results in 1. If you look at the truth table for the OR logic, let's set B to be always 1, so now, as long as the other operand is fixed to 1, the output will always be 1 regardless of what A is.



Boolean theorems: Single variable

- For single variables: Let's say **A** is a variable

Idempotent theorem

- **$AA = A$** (Any variable ANDed to itself is itself)
- **$A + A = A$** (Any variable ORed to itself is itself)

AND logic

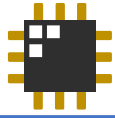
A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1

OR logic

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1

11

$AA = A$: It states that any variable ANDed with itself results in the variable itself. If you look at the truth table for the AND logic, let's set B to be always equal to A, so now, as long as the other operand is the same as A, the output just copies whatever A is.



Boolean theorems: Single variable

- For single variables: Let's say **A** is a variable

Idempotent theorem

- $AA = A$ (Any variable ANDed to itself is itself)
- $A + A = A$ (Any variable ORed to itself is itself)

AND logic

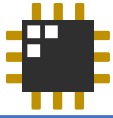
A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1

OR logic

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1

12

$A + A = A$: It states that any variable ORed with itself results in the variable itself. If you look at the truth table for the OR logic, let's set B to be always equal to A, so now, as long as the other operand is the same as A, the output just copies whatever A is.



Boolean theorems: Single variable

- For single variables: Let's say **A** is a variable

AND logic

A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1

OR logic

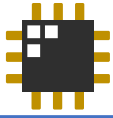
A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1

Involution theorem

- $(A')' = A$ (Complementing twice is itself)

13

$(A')' = A$: It states that the complement of the complement of a variable is the variable itself. If you look at the truth table for the NOT logic, let's apply the NOT operation twice on A, so now, the output just copies whatever A is.



Boolean theorems: Single variable

- For single variables: Let's say **A** is a variable

Complement theorem

- $A(A') = 0$ (ANDing to its complement is 0)
- $A + A' = 1$ (ORing to its complement is 1)

AND logic

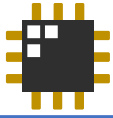
A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1

OR logic

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1

14

$A(A') = 0$: It states that a variable ANDed with its complement results in 0. If you look at the truth table for the AND logic, let's set B to be always the complement of A, so now, as long as the other operand is the complement of A, the output will always be 0.



Boolean theorems: Single variable

- For single variables: Let's say **A** is a variable

Complement theorem

- $A(A') = 0$ (ANDing to its complement is 0)
- $A + A' = 1$ (ORing to its complement is 1)

AND logic

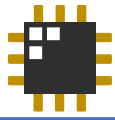
A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1

OR logic

A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1

15

$A + A' = 1$: It states that a variable ORed with its complement results in 1. If you look at the truth table for the OR logic, let's set B to be always the complement of A, so now, as long as the other operand is the complement of A, the output will always be 1.



Boolean theorems: Single variable

- For single variables: Let's say **A** is a variable

- $A(1) = A$ (Any variable ANDed to 1 is itself)
- $A + 0 = A$ (Any variable ORed to 0 is itself)

Identity theorem

- $A(0) = 0$ (Any variable ANDed to 0 is 0)
- $A + 1 = 1$ (Any variable ORed to 1 is 1)

Null theorem

- $AA = A$ (Any variable ANDed to itself is itself)
- $A + A = A$ (Any variable ORed to itself is itself)

Idempotent theorem

- $(A')' = A$ (Complementing twice is itself)
- $A(A') = 0$ (ANDing to its complement is 0)
- $A + A' = 1$ (ORing to its complement is 1)

Involution theorem

Complement theorem

Recapping the theorems:

$A(1) = A$: This is the Identity Theorem for AND. It states that any variable ANDed with 1 results in the variable itself. If you look at the truth table for the AND logic, let's set B to be always 1, so now, as long as the other operand is fixed to 1, the output just copies whatever A is. Thus, it is true that $A(1) = A$.

$A + 0 = A$: This is the Identity Theorem for OR. It states that any variable ORed with 0 results in the variable itself. If you look at the truth table for the OR logic, let's set B to be always 0, so now, as long as the other operand is fixed to 0, the output just copies whatever A is. Thus, it is true that $A + 0 = A$.

$A(0) = 0$: This is the Null Theorem for AND. It states that any variable ANDed with 0 results in 0. If you look at the truth table for the AND logic, let's set B to be always 0, so now, as long as the other operand is fixed to 0, the output will always be 0 regardless of what A is. Thus, it is true that $A(0) = 0$.

$A + 1 = 1$: This is the Null Theorem for OR. It states that any variable ORed with 1 results in 1. If you look at the truth table for the OR logic, let's set B to be always 1, so now, as long as the other operand is fixed to 1, the output will always be 1 regardless of what A is. Thus, it is true that $A + 1 = 1$.

$AA = A$: This is the Idempotent Theorem for AND. It states that any variable ANDed

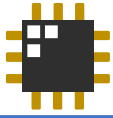
with itself results in the variable itself. If you look at the truth table for the AND logic, let's set B to be always equal to A, so now, as long as the other operand is the same as A, the output just copies whatever A is. Thus, it is true that $AA = A$.

$A + A = A$: This is the Idempotent Theorem for OR. It states that any variable ORed with itself results in the variable itself. If you look at the truth table for the OR logic, let's set B to be always equal to A, so now, as long as the other operand is the same as A, the output just copies whatever A is. Thus, it is true that $A + A = A$.

$(A')' = A$: This is the Theorem of Double Negation, also called involution. It states that the complement of the complement of a variable is the variable itself. If you look at the truth table for the NOT logic, let's apply the NOT operation twice on A, so now, the output just copies whatever A is. Thus, it is true that $(A')' = A$.

$A(A') = 0$: This is the Complement Theorem for AND. It states that a variable ANDed with its complement results in 0. If you look at the truth table for the AND logic, let's set B to be always the complement of A, so now, as long as the other operand is the complement of A, the output will always be 0. Thus, it is true that $A(A') = 0$.

$A + A' = 1$: This is the Complement Theorem for OR. It states that a variable ORed with its complement results in 1. If you look at the truth table for the OR logic, let's set B to be always the complement of A, so now, as long as the other operand is the complement of A, the output will always be 1. Thus, it is true that $A + A' = 1$.



Boolean theorems: Multiple variables

- For multiple variables: Let's say **A**, **B**, and **C** are variables
 - **$AB = BA$, $A+B = B+A$** (Commutativity)

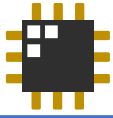
Commutativity theorem

The order of the operands does not matter

17

These are some of the fundamental theorems of Boolean algebra that are indeed similar to the laws in arithmetic. Let's go through them:

$AB = BA$, $A+B = B+A$: These are the Commutativity Theorems. They state that the order of the variables does not matter in an AND or OR operation, just like addition or multiplication in arithmetic.



Boolean theorems: Multiple variables

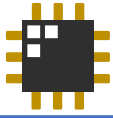
- For multiple variables: Let's say **A**, **B**, and **C** are variables
 - $AB = BA$, $A+B = B+A$ (Commutativity)
 - $(AB)C = A(BC)$, $(A+B)+C = A+(B+C)$ (Associativity)

Associativity theorem

The grouping of the operands does not matter

18

$(AB)C = A(BC)$, $(A+B)+C = A+(B+C)$: These are the Associativity Theorems. They state that the way variables are grouped in an AND or OR operation does not matter, similar to addition or multiplication in arithmetic.



Boolean theorems: Multiple variables

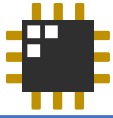
- For multiple variables: Let's say **A**, **B**, and **C** are variables
 - $AB = BA$, $A+B = B+A$ (Commutativity)
 - $(AB)C = A(BC)$, $(A+B)+C = A+(B+C)$ (Associativity)
 - $A(B+C) = AB + AC$ (Distributivity)

Distributivity theorem

AND distributes over OR

19

$A(B+C) = AB + AC$: This is the Distributivity Theorem. It states that AND distributes over OR, similar to how multiplication distributes over addition in arithmetic.



Boolean theorems: Multiple variables

- For multiple variables: Let's say **A**, **B**, and **C** are variables
 - $AB = BA$, $A+B = B+A$ (Commutativity)
 - $(AB)C = A(BC)$, $(A+B)+C = A+(B+C)$ (Associativity)
 - $A(B+C) = AB + AC$ (Distributivity)
 - $A+(BC) = (A+B)(A+C)$ (Distributivity, but weirder!)

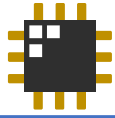
Distributivity theorem

OR distributes over AND

(This is the dual of the previous statement)

20

$A+(BC) = (A+B)(A+C)$: This is another form of the Distributivity Theorem. It's a bit less intuitive than the previous one since it doesn't work like this in math. What's really happening is that the OR operator is being distributed over the AND operator.



Boolean theorems: Multiple variables

- For multiple variables: Let's say **A**, **B**, and **C** are variables
 - $AB = BA$, $A+B = B+A$ (Commutativity)
 - $(AB)C = A(BC)$, $(A+B)+C = A+(B+C)$ (Associativity)
 - $A(B+C) = AB + AC$ (Distributivity)
 - $A+(BC) = (A+B)(A+C)$ (Distributivity, but weirder!)

Proof:

$$\begin{aligned}
 (A+B)(A+C) &= AA + AC + BA + BC && \text{(Distributivity)} \\
 \underline{AA} + AC + BA + BC &= \underline{A} + AC + AB + BC && \text{(Idempotent theorem)} \\
 \underline{A} + \underline{AC} + \underline{AB} + BC &= \underline{A}(1 + C + B) + BC && \text{(Distributivity, factor out)} \\
 A(\underline{1} + C + B) + BC &= A(1) + BC && \text{(Null theorem)} \\
 A(\underline{1}) + BC &= A + BC && \text{(Identity theorem)}
 \end{aligned}$$

21

This is legitimate though, and we can prove that this is indeed true.

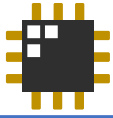
Applying the FOIL method that you know from math, we can perform the distribution and we can end up with $(AA + AC + BA + BC)$.

However, we know that A ANDed to itself is just itself, that's one of the theorems we showed earlier, thus this expression just becomes $(A + AC + AB + BC)$

Next, we can technically factor out the variable A. When factoring A out, note that there is a '1' that will be left from this single term. This is not new, and even in math it works like this, so what we are doing are intuitive so far.

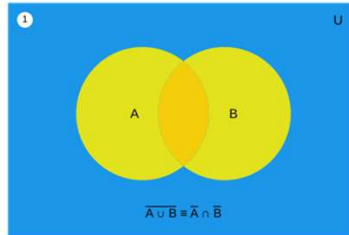
Now, for the expression $A(1+C+B) + BC$, we note that anything ORed to 1 will reduce to 1. This is also one of the theorems we showed earlier.

Therefore, we will end up with A ANDed to 1, which is just itself again that's one of the theorems earlier, giving us $A+BC$. This is equivalent to whatever we started with, indicating logical equivalence.



Boolean theorems: Multiple variables

- For multiple variables: Let's say **A**, **B**, and **C** are variables



- $(A+B)' = A'B'$

(De Morgan's)

- $(AB)' = A' + B'$

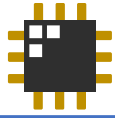
(De Morgan's)

De Morgan's theorem

'Distributing' the negation over OR

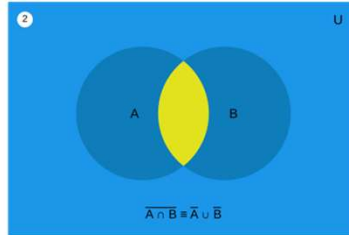
22

$(AB)' = A' + B'$, $(A+B)' = A'B'$: These are De Morgan's Theorems. They state that the complement of an AND operation is the same as the OR of the complements, and vice versa. The AND operation changes to OR (and vice versa) when taking the complement. This is a unique property of Boolean algebra and doesn't have a direct equivalent in arithmetic.



Boolean theorems: Multiple variables

- For multiple variables: Let's say **A**, **B**, and **C** are variables



- $(A+B)' = A'B'$

(De Morgan's)

- $(AB)' = A' + B'$

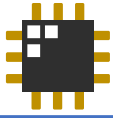
(De Morgan's)

De Morgan's theorem

'Distributing' the negation over AND

23

$(AB)' = A' + B'$, $(A+B)' = A'B'$: These are De Morgan's Theorems. They state that the complement of an AND operation is the same as the OR of the complements, and vice versa. The AND operation changes to OR (and vice versa) when taking the complement. This is a unique property of Boolean algebra and doesn't have a direct equivalent in arithmetic.



Boolean theorems: Multiple variables

- For multiple variables: Let's say **A**, **B**, and **C** are variables
 - $AB = BA, A+B = B+A$ (Commutativity)
 - $(AB)C = A(BC), (A+B)+C = A+(B+C)$ (Associativity)
 - $A(B+C) = AB + AC$ (Distributivity)
 - $A+(BC) = (A+B)(A+C)$ (Distributivity, dual)
- $(A+B)' = A'B'$ (De Morgan's)
- $(AB)' = A' + B'$ (De Morgan's)

24

A quick recap:

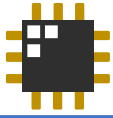
1. $AB = BA, A+B = B+A$: These are the Commutative Laws. They state that the order of the variables does not matter in an AND or OR operation, just like addition or multiplication in arithmetic.

2. $(AB)C = A(BC), (A+B)+C = A+(B+C)$: These are the Associative Laws. They state that the way variables are grouped in an AND or OR operation does not matter, similar to addition or multiplication in arithmetic.

3. $A(B+C) = AB + AC$: This is the Distributive Law. It states that AND distributes over OR, similar to how multiplication distributes over addition in arithmetic.

4. $A+(BC) = (A+B)(A+C)$: This is another form of the Distributive Law. It's a bit less intuitive than the previous one since it doesn't work like this in math. What's really happening is that the OR operator is being distributed over the AND operator.

5. $(AB)' = A' + B', (A+B)' = A'B'$: These are De Morgan's Laws. They state that the complement of an AND operation is the same as the OR of the complements, and vice versa. The AND operation changes to OR (and vice versa) when taking the complement. This is a unique property of Boolean algebra and doesn't have a direct equivalent in arithmetic.



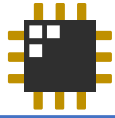
Boolean theorems: Multiple variables

- Other theorems can be derived from what we already know
 - **Covering theorem:** $A(A+B) = ?$

25

Let's simplify these expressions using the Boolean algebra theorems:

1. $A(A+B) = ?$ We can use the Distributive Law here, which states that $A(B+C) = AB + AC$. Applying this law to the expression $A(A+B)$, we get $AA + AB$. But we know that $(AA = A)$, the term AA can be simplified to A . So, we have $A + AB$. Continuing further, we can factor out A , we have: $A(1+B)$. But we know that $1+B = 1$. Therefore, we end up with $A(1) = A$. The simplified expression is just A .



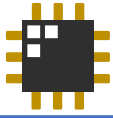
Boolean theorems: Multiple variables

- Other theorems can be derived from what we already know
 - **Covering theorem:** $A(A+B) = ?$
 - $AA + AB$ (Distributivity)
 - $A + AB$ (Idempotent)
 - $A(1+B)$ (Distributivity, factor out)
 - $A(1)$ (Null)
 - A (Identity)
- The dual should also be true: $A(A+B) \rightarrow A + AB = A$

26

Let's simplify these expressions using the Boolean algebra theorems:

1. $A(A+B) = ?$ We can use the Distributive Law here, which states that $A(B+C) = AB + AC$. Applying this law to the expression $A(A+B)$, we get $AA + AB$. But we know that $(AA = A)$, the term AA can be simplified to A . So, we have $A + AB$. Continuing further, we can factor out A , we have: $A(1+B)$. But we know that $1+B = 1$. Therefore, we end up with $A(1) = A$. The simplified expression is just A .

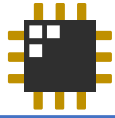


Boolean theorems: Multiple variables

- Other theorems can be derived from what we already know
 - **Combining theorem:** $AB + AB' = ?$

27

For this first one, we can perform distribution, putting the variable A inside the parenthesis. We will end up with $AA + AB$. But we know that $(AA = A)$, therefore the term AA can be simplified to A . So, we have $A + AB$. Continuing further, we can factor out A, giving us $A(1+B)$. But we know that anything ORed to 1 is just 1 from the previous theorems before, therefore $1+B = 1$. We end up with $A(1) = A$. This is the simplified expression.



Boolean theorems: Multiple variables

- Other theorems can be derived from what we already know
 - **Combining theorem:** $AB + AB' = ?$
 - $A(B+B')$ (Distributivity, factor out)
 - $A(1)$ (Complement)
 - A (Identity)
 - The dual should also be true: $AB + AB' \rightarrow (A+B)(A+B') = A$
Try proving it!

28

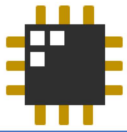
For this first one, we can perform distribution, putting the variable A inside the parenthesis. We will end up with $AA + AB$. But we know that $(AA = A)$, therefore the term AA can be simplified to A . So, we have $A + AB$. Continuing further, we can factor out A, giving us $A(1+B)$. But we know that anything ORed to 1 is just 1 from the previous theorems before, therefore $1+B = 1$. We end up with $A(1) = A$. This is the simplified expression.

$$AA + AB + AB' + BB'$$

$$A + AB + AB' + 0$$

$$A(1 + B + B')$$

$$A$$



Operator precedence

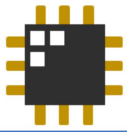
- **Operator precedence** is an ordering of logical operators designed to allow the dropping of parentheses in logical expressions.
- Below is a hierarchy of precedence for Boolean operators:
 1. Parenthesis (highest precedence)
 2. NOT
 3. AND
 4. OR (lowest precedence)

29

Just like in algebra, we also have a concept of operator precedence. This helps us understand which operations are performed first.

Here's the order of precedence from highest to lowest:

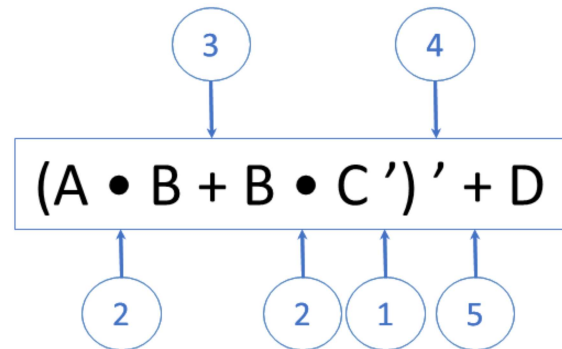
1. Parentheses – Always take the highest priority. Anything inside parentheses gets evaluated first.
2. NOT (') – Negation comes next. We take the complement of any variable.
3. AND (•) – Next, AND is performed.
4. OR (+) – Finally, OR has the lowest precedence, hence performed last.



Operator precedence

Example:

1. In the expression on the right, we prioritize the **expression inside the parentheses**
2. Inside the parentheses, the **NOT operation** goes 1st
3. 2nd are the **AND operations**
4. Then, 3rd is the **OR operation**
5. The parenthesis has a **NOT outside**, so that's the 4th
6. Finally, the **OR outside** is the 5th.



30

Just to illustrate the operator precedence, let's have the expression on the right.

First, we start with the parentheses, as they have the highest precedence. Everything inside must be evaluated before moving on.

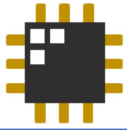
Within the parentheses, the NOT (') operation takes priority, so we first compute C '.

Next, we evaluate the AND (•) operations, solving $A \bullet B$ and $B \bullet C'$ separately.

After that, we apply the OR (+) operation, combining the results of the AND operations.

Now that we have a single expression inside the parentheses, we take its complement (').

Finally, we OR the result with D, completing the evaluation.



Boolean expression minimization

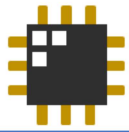
Simplify the expression:

$$A'B'(AC')'(B'C)'$$

31

Now, let's have a practice. Let's simplify the expression:

$$A'B'(AC')'(B'C)'$$



Boolean expression minimization

Simplify the expression:

$$A'B'(AC')'(B'C)'$$

$$= A'B'(A'+C)(B+C')$$

(De Morgan's)

$$= A'(A'+C)B'(B+C')$$

(Commutative)

$$= (A'A'+A'C)(B'B+B'C')$$

(Distributive)

$$= (A'+A'C)(0+B'C')$$

(Idempotent, Complement)

$$= A'(1+C)(B'C')$$

(Distributivity, Identity)

$$= A'(B'C')$$

(Null Element, Identity)

$$= A'B'C'$$

(Associativity)

Note: this is just one way of doing it. As long as you are following the theorems correctly, the final answer should be the same

32

$A'B'(A'+C)(B+C')$ Here, we're using De Morgan's Law to convert negations into OR operations.

$A'(A'+C)B'(B+C')$ Here, we rearrange the terms using the Commutative property.

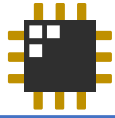
$(A'A'+A'C)(B'B+B'C')$ Here, we are using the Distributive property, resulting to these two expressions ANDed together.

$(A'+A'C)(0+B'C')$ Here, Idempotent and Complement laws simplified the redundant expressions.

$A'(1+C)(B'C')$ Here, we are using the Distributive property in reverse to factor out common terms and Identity property to remove 0 from the expression.

$A'(B'C')$ Here, we are using the Null law to simplify $(1+C) = 1$, resulting to $A'(1)(B'C')$ and Identity law to remove 1 from the expression.

$A'B'C'$ Here, we are removing the parentheses using the Associative property.



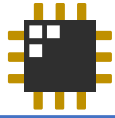
Boolean expression minimization

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

33

Let's have another practice. This is a different Boolean expression from before.

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$



Boolean expression minimization

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

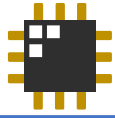
$$Q = A'B'(C' + C) + A'B(C' + C) + AB(C' + C)$$

$$Q = A'B' + A'B + AB$$

34

$Q = A'B'(C' + C) + A'B(C' + C) + AB(C' + C)$ Here, we're using the Distributive Law in reverse to factor out common terms. In each group of terms, $A'B'$, $A'B$, and AB are common, so we factor them out.

$Q = A'B' + A'B + AB$ Here, we're using the Identity Law for OR, which states that any variable ORed with its complement results in 1. So, $(C' + C)$ simplifies to 1, and any variable ANDed with 1 is itself, so $A'B'(1)$ simplifies to $A'B'$, and so on.



Boolean expression minimization

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

$$Q = A'B'(C' + C) + A'B(C' + C) + AB(C' + C)$$

$$Q = A'B' + A'B + AB$$

$$Q = A'B' + A'B + A'B + AB$$

$$Q = A'B' + A'B + A'B + AB$$

$$Q = A'(B' + B) + B(A' + A)$$

$$Q = A' + B \quad \text{Final answer}$$

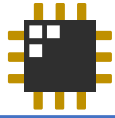
35

$Q = A'B' + A'B + A'B + AB$ Here, we replicate $A'B$. This is fine because we know that any expression ORed to itself is itself. Replicating $A'B$ is also helpful because we can perform further factoring out, given the common terms A' and B on the first and last terms.

$Q = A'(B' + B) + B(A' + A)$ Here, we're using the Distributive Law in reverse again to factor out common terms. In the first three terms, A' is common, so we factor it out. In the last two terms, B is common, so we factor it out.

$Q = A' + B$ Finally, we're using the Identity Law for OR again to simplify $(B' + B)$ and $(A' + A)$ to 1. So, $A'(1)$ simplifies to A' , and $B(1)$ simplifies to B .

So, the original Boolean expression $Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$ simplifies to $Q = A' + B$. This is the power of Boolean algebra in simplifying complex logical expressions!



Logical equivalence

- Are we sure that the simplified expression is equivalent to the original expression?
 - As long as you followed the theorems correctly, it should 😊



- You can write the truth table again using the simplified expression. The resulting truth table should be the same!

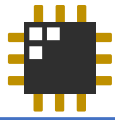
GIF from tenor.com

36

Of course, the next question is, after doing all the simplification that we did, are we still sure that the simplified expression is equivalent to the original expression?

Well, as long as you followed the theorems correctly, and you are not careless when applying them, then yes, they should be equivalent.

One way to verify the equivalence is to write the truth tables for both original and simplified expressions. The resulting truth table should be the same!



Logical equivalence

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

$$Q = A' + B$$

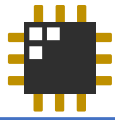
A	B	C	Q
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

37

Here's the other expression that we derived and the corresponding truth table. Let's verify that the simplified expression is indeed the same as the original expression.

A' = as long as A is 0, doesn't matter what B and C are, the output is 1.

B = as long as B is 1, doesn't matter what A and C are, the output is 1.



Logical equivalence

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

*Tables are the same
Logically equivalent!*

$$Q = A' + B$$

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

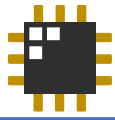
38

A' = as long as A is 0, doesn't matter what B and C are, the output is 1 (orange highlight)

B = as long as B is 1, doesn't matter what A and C are, the output is 1. (blue highlight)

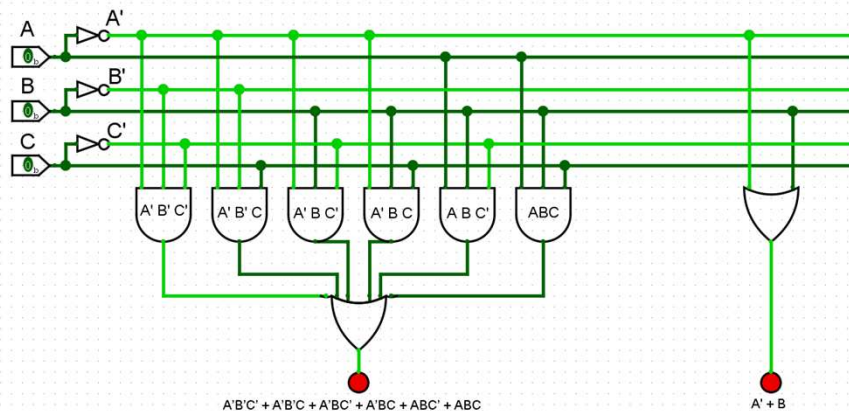
We also note that there's an intersection between the orange and blue highlights. Meaning the two terms we are considering has two rows in the truth table that they both correspond to. This is fine, since they both map to an output of 1 anyway.

As we can see, the resulting truth table for the simplified expression is indeed the same as the truth table of the original expression, thus proving their equivalence!



Logical equivalence

- If the truth tables are the same, they are functionally equivalent!
- Simulation demo:



**Fewer logic gates.
More efficient!**

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

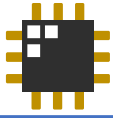
$$Q = A' + B$$

39

As a final note, we can also perform some simulations to really verify their equivalence. I have a simulator here where you can see the expressions being wired up. You can see the AND gates, OR gates, and NOT gates in there that exactly corresponds to how the expressions describe it. Try observing the connections on your own to verify!

On the left is the original expression and its corresponding logic diagram. On the right is the simplified expression and its corresponding diagram. (Simulator: note that when we are changing the inputs to any possible combination, the outputs shown in red behave similarly. This clearly indicates that they behave exactly the same. This further proves that they are logically equivalent.)

Moreover, as you can see, the simplified expression has lower number of logic gates. This actually alludes to that being more efficient in terms of hardware, since you can implement the same logic functionality while having fewer logic gates to do it. That is the ultimate benefit of Boolean logic simplification.



Forms of Boolean expressions

- Truth tables are unique but logic expressions and diagrams are not
 - Two Boolean expressions are equal if they have the same truth table
- We can manipulate a Boolean expression to change its *form*.
- Some common forms of Boolean expressions:
 - **Sum-of-Products (SOP)** and **Product-of-Sums (POS)**
 - Sum of *minterms* and Product of *maxterms*

40

Logic expressions and circuit diagrams are not unique because Boolean algebra allows expressions to be manipulated into different forms while preserving their function, just like what we did previously.

Moving on to the common forms of Boolean expressions, we have:

Sum-of-Products (SOP) expression is a sum (OR) of multiple product (AND) terms. Each term is a product of literals, and the final expression is a sum of these products.

Product-of-Sums (POS) expression is a product (AND) of multiple sum (OR) terms. Each term is a sum of literal, and the final expression is a product of these sums, hence called Product-of-Sums.

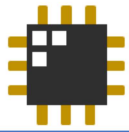
A **minterm** is a product (**AND**) of all input variables, while a **maxterm** is a sum (**OR**) of all input variables. In both cases, each variable appears in either its true or complemented form.

Now, how do we use them to express Boolean functions?

In the **Sum of Minterms** form, we take the minterms corresponding to the rows in the truth table where the output is **1** and OR them together.

In the **Product of Maxterms** form, we take the maxterms corresponding to the rows in the truth table where the output is **0** and AND them together.

These common forms will be discussed in detail later.



Definition of relevant terms

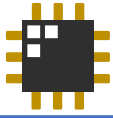
Term	Definition	Examples
Literal	Variable or its complement	A, A', B, B'
Product Term	Series of literals related by AND	$AB'D$ $ACD'E$
Sum Term	Series of literals related by OR	$A+C+D'$ $A+B+D'+E$
Normal Term	Product or sum term in which no variable appears more than once; In a sense, normal form is the “best” form	$A+C+D'$ BCD

Literals are the simplest components of Boolean expressions. They can either be a variable itself or the complement of a variable. These literals are used to construct more complex Boolean expressions.

Product term is formed by ANDing multiple literals together. This means all the variables in the term must be true for the term to evaluate to 1. Product terms are important in the SOP form, where multiple product terms are ORed together.

Sum term is formed by ORing multiple literals together. This means that if any one of the literals is true, the entire sum term evaluates to 1. Sum terms are used in POS form, where multiple sum terms are ANDed together.

Normal term is either a product term or a sum term, but with important condition: no variable appears more than once in the term. Normal form is considered the best form because it is already optimized and avoids redundancy.



Forms of Boolean expressions

- Sum-of-Products (SOP) Form: product terms ORed together

$$ABC + A'B'C + B$$

- Product-of-Sums (POS) Form: sum terms ANDed together

$$(A+B)(A+B'+C)C$$

42

Again, SOP is formed by ANDing literals together to create product terms, then ORing these product terms together. For example:

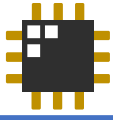
$$ABC + A'B'C + B$$

Here, we have three product terms: ABC , $A'B'C$, and B . These terms are ORed together, which makes this an SOP expression.

POS is the opposite of SOP, it is formed by ORing literals to create sum terms, then ANDing those sum terms together. For example:

$$(A+B)(A+B'+C)C$$

We have three sum terms: $(A+B)$, $(A+B'+C)$, and C . These terms are ANDed together, making this a POS expression.



Canonical forms

- It is useful to specify Boolean functions in a form that:
 - Allows comparison for equality
 - Has direct correspondence with their truth tables
- Commonly used canonical forms:
 - **Sum of minterms**, or **Standard Sum-of-Products (SSOP)**
 - **Product of maxterms**, or **Standard Product-of-Sums (SPOS)**
- A Boolean expression in **standard** form has all input literals appearing in each product or sum term

43

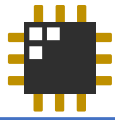
When working with Boolean functions, it's useful to write them in a standardized form that allows easy comparison and direct mapping to their truth tables.

As we have mentioned earlier:

In **Sum of Minterms** or **Standard Sum-of-Products (SSOP)** form, we take the minterms corresponding to the rows in the truth table where the output is **1** and OR them together.

In the **Product of Maxterms** or **Standard Product-of-Sums (SPOS)** form, we take the maxterms corresponding to the rows in the truth table where the output is **0** and AND them together.

We'll go through an example later to help you visualize how this works.



Recall: Truth tables to Boolean expression

A	B	C	Q	
0	0	0	1	A'B'C'
0	0	1	1	A'B'C
0	1	0	1	A'BC'
0	1	1	1	A'BC
1	0	0	0	
1	0	1	0	
1	1	0	1	ABC'
1	1	1	1	ABC

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

This is actually written in
Standard Sum-of-Products
(SSOP)!

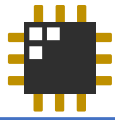
Note how each term contains all literals

44

- For the first row, A must be 0, B must be 0, and C must be 0 to make the output Q equal to 1. This is equivalent to A'B'C'.
- For the second row, A must be 0, B must be 0, and C must be 1 to make the output Q equal to 1. This is equivalent to A'B'C.
- For the third row, A must be 0, B must be 1, and C must be 0 to make the output Q equal to 1. This is equivalent to A'BC'.
- For the fourth row, A must be 0, B must be 1, and C must be 1 to make the output Q equal to 1. This is equivalent to A'BC.
- For the seventh row, A must be 1, B must be 1, and C must be 0 to make the output Q equal to 1. This is equivalent to ABC'.
- For the eighth row, A must be 1, B must be 1, and C must be 1 to make the output Q equal to 1. This is equivalent to ABC.

Combining all these terms using the OR operation, we get the Boolean expression

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$



Sum of minterms expression

	A	B	C	Q	
0	0	0	0	1	$A'B'C'$
1	0	0	1	1	$A'B'C$
2	0	1	0	1	$A'BC'$
3	0	1	1	1	$A'BC$
4	1	0	0	0	
5	1	0	1	0	
6	1	1	0	1	ABC'
7	1	1	1	1	ABC

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

This can also be written as:

$$\sum m(0, 1, 2, 3, 6, 7)$$

Sum of minterms

(minterms: rows (binary equivalent) where the output is 1)

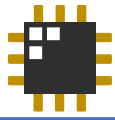
Here, we are using the same expressions we got on the last slide.

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

Now, instead of writing out the full Boolean expression, we can represent it more concisely using sum of minterms:

$$Q = \sum m(0, 1, 2, 3, 6, 7)$$

Where Σ indicates sum of minterms, and $m(0,1,2,3,6,7)$ lists the row numbers in the truth table where function is 1.



Standard Product-of-Sums

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

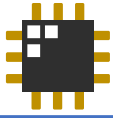
Writing the **Standard Product-of-Sums (SPOS)** expression requires us to look at the rows where the output is 0.

____?
____?

46

When writing the Standard Product-of-Sum (SPOS) expression, we focus on the rows in the truth table where the output is 0, opposite to the Sum of Minterms (SSOP) form.

Now, what do you think should the Boolean function be for this case?



Standard Product-of-Sums

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Writing the **Standard Product-of-Sums (SPOS)** expression requires us to look at the rows where the output is 0.

$$Q = AB'C' + AB'C ?$$

$AB'C' ?$

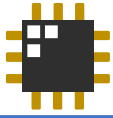
$AB'C ?$

No!

This expression tells us when $Q = 1$

47

So, what's wrong here? The problem is that this expression follows the Standard Sum-of-Product (SSOP) form, not SPOS! If we use this expression as it is, the output for these terms will be $Q = 1$, which is incorrect.



Standard Product-of-Sums

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

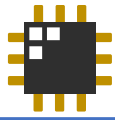
Writing the **Standard Product-of-Sums (SPOS)** expression requires us to look at the rows where the output is 0.

$$Q = (AB'C' + AB'C)'$$

This expression tells us when **Q = 1**

48

To correct this mistake, we need to take the complement of the expression. This will allow us to properly convert it into the SPOS form, ensuring that it represents the correct logic.



Standard Product-of-Sums

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Writing the **Standard Product-of-Sums (SPOS)** expression requires us to look at the rows where the output is 0.

$$Q = (AB'C' + AB'C)'$$

$$Q = (AB'C')'(AB'C)'$$

$$Q = (A'+B+C)(A'+B+C')$$

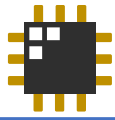
This expression tells us when Q = 1

49

$Q = (AB'C')'(AB'C)'$ Here, we will apply De Morgan's Theorem, where negating a sum turns it into a product of negations. Now, instead of sum of terms, we now have a product of terms, which is exactly what we want in SPOS!

$Q = (A'+B+C)(A'+B+C')$ Here, we individually apply De Morgan's Theorem to each term: $(AB'C')' = A' + B + C$ and $(AB'C)' = A' + B + C'$,

We now have the final SPOS expression!



Standard Product-of-Sums

A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

$$Q = (A' + B + C)(A' + B + C')$$

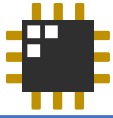
This is the Standard Product-of-Sums (SPOS)!

Note how each term contains all literals

(A' + B + C) The expressions per row are
(A' + B + C') **duals** of the original SOP term

50

Now, we have the correct form! Notice how each term include all literals and the expressions per row are the duals of the original SOP term.



Sum of minterms expression

	A	B	C	Q
0	0	0	0	1
1	0	0	1	1
2	0	1	0	1
3	0	1	1	1
4	1	0	0	0
5	1	0	1	0
6	1	1	0	1
7	1	1	1	1

$$Q = (A' + B + C)(A' + B + C')$$

This can also be written as:

$$\prod M(4, 5)$$

(A' + B + C)

(A' + B + C')

Product of maxterms

(maxterms: rows (binary equivalent) where the output is 0)

51

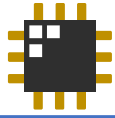
Here, we are using the same expressions we got on the last slide.

$$Q = (A' + B + C)(A' + B + C')$$

Now, instead of writing out the full Boolean expression, we can represent it more concisely using product of maxterms:

$$Q = \prod M(4, 5)$$

Where $\prod$ indicates product of maxterms, and $M(4,5)$ lists the row numbers in the truth table where function is 0.



Summary: Canonical forms

Standard Sum-of-Products (SSOP) **Standard Product-of-Sums (SPOS)**

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC$$

$$Q = (A'+B+C)(A'+B+C')$$

$$\sum m(0, 1, 2, 3, 6, 7)$$

Sum of minterms

$$\prod M(4, 5)$$

Product of maxterms

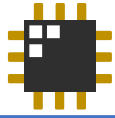
Both are referring to the same truth table!

Minterms and maxterms are *mutually exclusive*

52

Now that we both have SSOP and SPOS expressions, it's important to note that both forms describe the same truth table and that the minterms and maxterms are mutually exclusive, meaning minterms focus on where the output is 1, while maxterms focus on where the output is 0.

Even though they look different, SSOP and SPOS are just two different ways of expressing the same logic function.



Logical equivalence

- Of course, SSOP and SPOS are **logically equivalent!**
 - They correspond to the same truth table!

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC \quad \textbf{SSOP}$$

$$\boxed{Q = A' + B} \text{ (from the minimization process shown earlier)}$$

$$Q = (A' + B + C)(A' + B + C') \quad \textbf{SPOS}$$

Prove that this is equal to $Q = A' + B$

53

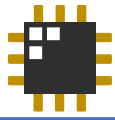
$Q = A'A' + A'B + A'C' + BA' + BB + CC' + CA' + CB + CC'$ Here, we expanded the expression using the distributive property.

$Q = A' + A'B + A'C + A'B + B + 0 + A'C + BC + 0$ Here, we apply idempotent law for $A'A'$ and BB and complement law for CC'

$Q = A'(1 + B + C + B + C) + B(1 + C)$ Here, we use the distributive property in reverse to factor out A' and B .

$Q = A' + B$ Here, we apply Null law to simplify $(1+B+C+B+C)$ and $(1+C)$

Here, we can see that both the SSOP and SPOS are logically equivalent!



Logical equivalence

- Of course, SSOP and SPOS are **logically equivalent!**
 - They correspond to the same truth table!

$$Q = A'B'C' + A'B'C + A'BC' + A'BC + ABC' + ABC \quad \textbf{SSOP}$$

$$\boxed{Q = A' + B} \text{ (from the minimization process shown earlier)}$$

$$Q = (A' + B + C)(A' + B + C') \quad \textbf{SPOS}$$

$$Q = A'A' + A'B + A'C' + BA' + BB + CC' + CA' + CB + CC'$$

$$Q = A' + A'B + A'C' + A'B + B + 0 + A'C' + BC + 0$$

$$Q = A'(1 + B + C + B + C) + B(1 + C)$$

$$\boxed{Q = A' + B}$$

54

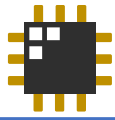
$Q = A'A' + A'B + A'C' + BA' + BB + CC' + CA' + CB + CC'$ Here, we expanded the expression using the distributive property.

$Q = A' + A'B + A'C' + A'B + B + 0 + A'C' + BC + 0$ Here, we apply idempotent law for $A'A'$ and BB and complement law for CC'

$Q = A'(1 + B + C + B + C) + B(1 + C)$ Here, we use the distributive property in reverse to factor out A' and B .

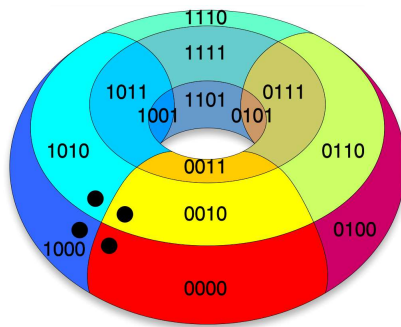
$Q = A' + B$ Here, we apply Null law to simplify $(1+B+C+B+C)$ and $(1+C)$

Here, we can see that both the SSOP and SPOS are logically equivalent!



Teaser for next lecture: K-maps

- How do we know if it's already in the most simplified expression?
 - By leveraging patterns in Boolean expressions and how they simplify, we can do a *visual inspection of the truth table* **to determine the simplest form**



0000	0100	1100	1000
0001	0101	1101	1001
0011	0111	1111	1011
0010	0110	1110	1010

It looks like this...
Though probably not drawn as a doughnut ☹

Image credit: "Karnaugh map" wikipedia.org

55

By recognizing patterns in Boolean expressions and understanding how they simplify, we can visually inspect the truth table. However, manually simplifying complex expressions can be tedious and error-prone.

This is where **Karnaugh Maps (K-Maps)** come in! K-Maps provide a structured way to group terms and find the simplest expression efficiently. In the next lecture, we'll explore how to use K-Maps to minimize Boolean functions with ease.